COMP.SE.140: Basics of containers and microservices

Report by: Emilija Nikita [S.N. 153804878]

22.09.2025

Table of contents

List of Figures

List of Tables

1. Introduction

Task definition. In the exercise a simple system composed of three small services is built.

The must be implemented while using different programming languages for separate services (**service1** and **service2**). There is also a third service, used for shared persistent storage, introduced - **storage**.

Each service is specified in a table down below:

Table 1. Simplified service specifications

<table>
<tr><th colspan="4">Service1</th></tr>
<tr><td>The only service that can be accessed externaly (via browser or curl command)</td><td>Analyses its state and creates a record in persistent shared memory*</td><td>Works as a proxy – forwards received request to other services:
/status => Service2
/log => Storage</td><td>Logs the record to two alternative persistent storages:
- Storage-service
-Volume implemented vStorage</td></tr>
<tr><th colspan="4">Service2</th></tr>
<tr><td colspan="2">Analyses its state and creates a record in persistent shared memory*</td><td colspan="2">Logs the record to two alternative persistent storages:
- Storage-service
-Volume implemented vStorage</td></tr>
<tr><th colspan="4">Storage</th></tr>
<tr><td colspan="2">HTTP POST /log => append the incoming record in persistent memory</td><td colspan="2">HTTP GET /log => get the whole log content</td></tr>
</table>

* - State analysis consists of creating the following record:

```
timestamp: uptime <x> hours, free disk in root: <x> mbytes
```

2. Platform Information

This section describes the hardware, operating system and software versions used during the exercise.

CPU:	11th Gen Intel(R) Core(TM) i5-1155G7 @ 2.50
RAM:	15Gi
Disk size:	47 GB
Virtualization:	none
Operating System:	Ubuntu 22.04.5 LTS (Jammy Jellyfish)
Docker version:	Docker version 28.4.0, build d8eb465
Docker-compose version:	docker-compose version 1.29.2

3. Architecture Diagram

The application's architecture consists of three main components: **Service1** (*Rust*), **Service2** (*Nim*) and **Storage** (*Node.js*). They are interconnected via an internal Docker network. Both **Service1** and **Service2** are application containers which communicate with each other and log status records into shared persistent storages. There are two ways the storages were implemented: a) bind-mounted host directory (./vstorage) and b) a named Docker volume in a form of the third service (**Storage**). In the diagram (Figure 1), the arrows represent following interactions:

- Green arrow between vStorage volume and ./vstorage directory symbolizes data flow to persistance storage and host;
- Black solid arrows show interactions with other services or volumes. Some of them have a title, which represent the endpoints to access services;
- Black dashed arrows symbolize data responses;
- The blue arrows mean interactions between external API and services. Since the task specified only **Service1** should be accessible through external means, this kind of arrow connects external API and this service.

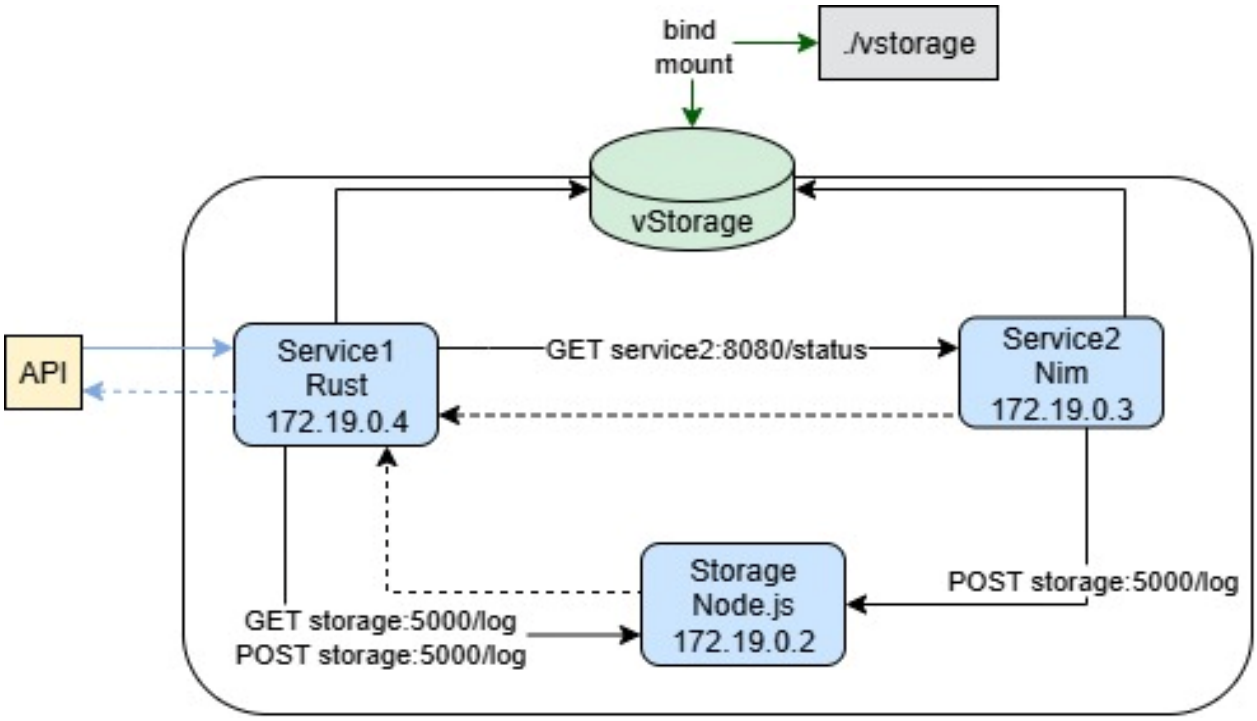

Figure 1. Architecture diagram

4. Analysis of Status Records

For this assessment, uptime was measured as the difference between when the service was first started and request time. The starting time was saved into a variable and used to calculate the uptime in hours. For clarification purposes, it was decided to use five decimal places.

Regarding disk space, the root directory of *Linux* (/) was selected for the data record. Since I have a dual-boot for both *Windows* and *Ubuntu* configured, I had the chance to compare reported values between the two environments.

It appeared that when running the containers in Windows environment, the free root disk space reported by the containers was almost nine times higher and greatly surpassed the actual *Windows* SSD capacity (which is 476.94 GB). The reason it might have happened is because *Docker Desktop* runs containers inside a virtual Linux machine (WSL2), so the containers' root filesystem is actually the virtual disk of that VM rather than the Windows host system. In contrast, the reported free root disk space in *Ubuntu* environment matched the actual system free space.

Interestingly, in the *Windows* environment, the mounted volume (/vstorage) reflected the host filesystem's free space accurately. This demonstrates that measurements taken on bind-

mounted directories are meaningful for monitoring host storage, while measurements on container root filesystem space reflect the Virtual Machine's virtual disk instead.

These measurements could become more relevant if the application explicitly recorded the free space of the host-mounted directories rather than relying on the root directory only. Additionally, tracking other metrics, such as memory usage or disk I/O could improve system monitoring.

5. Analysis of Persistent Storage Solutions

In this assessment, two different persistent storage solutions were implemented: a bind-mounted host directory (./vstorage) and a named Docker volume (storageData). This section contains comparison between the choices, including pros and cons of each.

Table 2. Pros and cons of different storage solutions

Pros	Cons
Bind-mount **vStorage**	
Data is directly accessible on host machine – easy to inspect, backup or modify files	Not portable – the path is specific to host environment
Useful for development and debugging – changes are immediately visible	Can cause permission issues if host directory permissions do not match the container user
Simple to set up and understand	Considered a bad practice for production, because containers are less isolated from the host.
Named Volume **Separate container**	
Portable – the volume is managed by Docker and can be easily manipulated	Not directly visible on the host, making debugging harder
Isolated from the host filesystem, reducing potential permission and security issues	Requires docker commands to clean or backup the data
Survives container recreation and restart automatically	Less transparent for learning purposes compared to bind-mount

Comparison:

Bind mounts are better for development and debugging, where immediate access to host files is helpful. It is possible to access files from the named volume inside the container but without Docker Desktop GUI it can be quite difficult to check, which might take additional time. On the other hand, named volumes are better for production or more isolated environments. That ensures data persistance across container lifecycles without relying on host-specific paths.

Considering clean-up, bind-mounted storage can be reset by simply deleting a file inside the file system. Named volumes need specific Docker commands for that.

To conclude everything, bind mounts are developer-friendly – the files are accessible and easily manipulated with outside Docker environment, the setup is simpler, thus working with them is quite an intuitive task. On the other hand, named volumes are more production-friendly, although they require additional work to set up properly and debug.

6. Teacher's Instruction for Clean-up

To clean the persistent storage, follow the steps down below:

1. **Bind-Mounted Directory:**

This storage is directly mapped to a folder inside the host machine. To clean it, simply delete the contents of the host directory:

```
rm -rf ./vstorage/*
```

2. **Named Volume [vStorage]:**

This storage is managed internally by Docker and does not appear in the host filesystem. To clean it up, use Docker Compose to remove the volume associated with the storage service:

```
sudo docker-compose down -v
```

Note: the commands above might require `sudo` depending on your Docker setup.

7. Difficulties and Problems

For the application's implementation, Rust, Nim and JavaScript were chosen as the main programming languages. This decision made the project development more interesting but also more challenging due to differences in frameworks and ecosystem maturity. Since it was my

first time developing RESTful applications in both Rust and Nim, I had to spend considerable time studying documentation, experimenting with examples, and selecting the proper tools for the stack. Nim in particular proved to be the most difficult in terms of web development, since its ecosystem for this matter is less mature compared to Rust or JavaScript.

When dealing with Dockerfiles, several issues arose. The package installation during image builds often failed unexpectedly. In many cases, the only way forward was to remove the failed build images and start again with the new adjustments (code refactoring, package imports, etc.). Frequent rebuilds of images slowed down the development process significantly. Additionally, many Docker commands required elevated privileges, which obligated me to re-run the same CLI commands with sudo, which was inconvenient and frustating. It still remains unresolved but perhaps will be taken care of in the nearest future.

Debugging containers and services also presented challenges. Luckily, I've chosen to first develop and test the application services separately before integrating them into a Dockerized environment. This approach led to an easier migration but did not eliminate difficulties. It was not straightforward to debug the services inside Docker containers. Tthis was especially noticeable when checking the internal endpoints with `curl`.

Another complication appeared in the measurement of disk space. On Windows, Docker reported much larger root filesystem size than the actual SSD size of the host machine, which created confusion while interpreting status records. On Ubuntu, however, the reported values matched the host's disk space, which highlighted the difference in how Docker Desktop manages resources on Windows compared to native Linux.

Finally, managing multiple services, storage solutions and ensuring proper communication between containers was more complex than anticipated. Setting up bind mounts and named volumes required experimentation to understand their behaviour, especially when files did not appear on the host system as expected (the issue in this instance was that after changing the source files, I did not correctly re-built the images, so it did not work).

8. Conclusion

This exercise provided hands-on experience in building a containerized microservice architecture with multiple programming languages, Docker networking and persistent storage solutions. It highlighted the practical differences between development-friendly and production-friendly storage approaches, as well as the challenges of debugging, managing service depenedencies and ensuring data persistance across environments.

Through experimentation on both Windows and Ubuntu, it became clear how host systems influence container behaviour, particularly in filesystem reporting. Despite difficulties with Dockerfiles, debugging and environment-specific issues, the project successfully demonstrated the core concepts of containers, microservices and persistent storage.

Overall, the work offered valuable insights of the pros anad cons of development with containerization and laid a strong foundation for more complex deployments in future projects.